

РАЗРАБОТКА С ПОМОЩЬЮ ИИ

Два робота и один скрипт на Bash

Юрий Панчуть · октябрь 2026 года · о проекте `basics-graphics-music`

Я не виртуоз Bash-скриптинга, но оценить качество кода на Bash я могу. Как выяснилось, этого достаточно, чтобы поставить полезный опыт: дать двум ИИ-агентам одну и ту же задачу в двух отдельных клонах одного репозитория, заставить каждого разобрать работу другого — и посмотреть, к чему они придут.

Сначала — о репозитории

`basics-graphics-music`, или BGM, — это коллекция переносимых примеров на SystemVerilog для проектирования на ПЛИС и для заказных микросхем. Берёте плату, запускаете один скрипт, чтобы её выбрать, и другой, который синтезирует проект, разместит и разведёт его и прошьёт плату. Нажимаете кнопки, смотрите, как загораются светодиоды, видите картинку на VGA или HDMI, слышите звук из аудиовыхода.

Смысл проекта в том, что примеры переносимы, а переносимыми их делает самая неблагодарная его часть: скрипты и обёртки для плат. Сейчас BGM поддерживает **54 разные платы**: платы Terasic серии DE, Digilent Arty, Nexys и Basys, очень большое семейство Sipeed Tang, iCEBreaker, OrangeCrab, Karnix, «Марсоход», ALINX, OMDAZZ и другие. За ними стоят пять разных наборов инструментов: Intel Quartus, AMD Vivado, Gowin EDA, инструменты Lattice и открытый поток на основе Yosys.

Эти 54 платы занимают **112 каталогов**, и разница между двумя числами сама по себе стоит отдельного абзаца, потому что в ней и состоит суть задачи. У платы редко бывает одна конфигурация. На одну только Tang Nano 9K приходится восемнадцать каталогов: с

модулем TM1638 (клавиатура плюс индикатор) и без него, с выводом на HDMI или на один из трёх разных ЖК-экранов, со сборкой штатными инструментами вендора или через Yosys, а в одном случае — с тактированием на 50 МГц через дополнительный PLL вместо родных 27. Ещё четырнадцать каталогов приходится на Tang Primer 20K Dock и по семь — на iCEBreaker и Tang Nano 20K.

Пятьдесят четыре платы, сто двенадцать конфигураций, пять наборов инструментов. Этот зверинец не нужно уничтожать — он и есть реальность преподавания цифровой схемотехники, а скрипты существуют, чтобы его спрятать.

Всё это важно из-за того, как проектом пользуются на практике. Когда я веду семинар, в аудитории не одна разновидность плат. Там то, что есть у университета, то, что принесли студенты, и то, что я привёз с собой. Студент с Tang Nano 9K и студент с Nexys A7 должны уметь запустить одну и ту же лабораторную из одного и того же репозитория одними и теми же двумя командами. В этом и состоит цель, и каждый час, потраченный на слой Bash-скриптов, возвращается сотней часов, которые другие люди не потратят на борьбу с инструментами.

Где это использовалось — в частности:

- 2022 — Американский университет в Центральной Азии, Бишкек, Киргизия
- 2023 — LaLambda, Тбилиси, Грузия
- 2024 — Университет ADA, Баку, Азербайджан; Hacker Dojo, Кремниевая долина
- 2025 — Автономный университет Нижней Калифорнии, Тихуана, Мексика; Российско-Армянский университет и Институт проблем информатики и автоматизации НАН РА, Ереван, Армения

Те же примеры использует [Школа синтеза цифровых схем](#), семинары которой проходят в **25 университетах России и Белоруссии**, — это самый массовый потребитель этих примеров.

Преподаватели берут BGM потому, что он убирает сложность вендорских инструментов из первой недели курса. Никто не узнаёт ничего нового о конечных автоматах за сорок минут возни с диалогами лицензий и структурой каталогов. А скрипт, падающий по

неочевидной причине, обходится не в одну поднятую руку в одной аудитории, а в один и тот же потерянный час сразу во всех.

Задача, которую нам подарила AMD

Теперь о самом скрипте. Много лет Vivado устанавливался в каталог, который выглядел так:

```
<родительский каталог>/Xilinx/Vivado/2023.1
```

Наш скрипт настройки искал именно это — в `$HOME`, `/opt` и `/tools`. Потом, начиная с версии 2024.2, AMD поставила номер версии *выше* названия продукта, да и каталог вендора теперь называется не обязательно Xilinx:

```
<родительский каталог>/AMD/2026.1/Vivado
```

Скрипт перестал находить Vivado. Хуже того, позже я узнал, что он делал кое-что более стыдное, чем просто не находить. Если установка была вообще без каталога версии, скрипт послушно перебирал внутренние подкаталоги самого Vivado, как будто это номера версий, сортировал их по алфавиту и выбирал последний:

```
XILINX_VIVADO=.../Xilinx/Vivado/tps  
PATH=...:.../Xilinx/Vivado/tps/bin
```

После чего уверенно печатал предупреждение, что установлено несколько версий Vivado, и перечислял среди них `tps`, `scripts` и `lib`. Студент, наступивший на это, получил бы сообщение о том, что `vivado` не найден в пути, — на несколько шагов позже и совсем не в том месте, где настоящая причина.

Эксперимент

У меня рядом лежат два клона репозитория: `~/claude` и `~/codex`. Я дал одну и ту же задачу Claude в одном и Codex в другом, в отдельных ветках, так что про существование друг друга они не знали. А потом несколько раз повторил три действия:

1. Попросить каждого реализовать или улучшить поиск.
2. Попросить каждого прочитать чужой клон и написать отчёт со сравнением двух реализаций — с достоинствами и недостатками каждой.
3. Попросить каждого улучшить свой код по итогам этого сравнения.

Вот здесь и перестаёт иметь значение то, что я не виртуоз Bash. Мне не нужно было придумывать правильную реализацию. Мне нужно было прочитать два довода об одном и том же коде и решить, который лучше, — а это навык, которым обладать гораздо проще и который гораздо проще применять быстро.

Первый раунд: они оказались разными

С раскладкой каталогов справились оба. Интересные различия были в другом, и каждый нашёл то, что упустил другой.

Codex обеспокоился случаем, который мне бы в голову не пришёл. Под Cygwin и MSYS — а ими пользуется немало студентов — командой `sort` в пути может оказаться виндовый `sort.exe`, который про сортировку версий не знает ничего. Поэтому Codex сначала пробует `/usr/bin/sort` и `/bin/sort`, а `sort` из пути — только в последнюю очередь. И он не просто проверяет, что опция `-V` принимается: он подсовывает кандидату два номера версий и смотрит на ответ.

```
printf '2026.10\tb\n2026.2\ta\n' | LC_ALL=C "$sort_candidate" -t '$\t' -k1,1V
# принять этот sort только если 2026.2 действительно вышло раньше 2026.10
```

Версия Claude проверяла только то, что `sort -V` не завершился ошибкой. Когда я попросил Claude протестировать собственный код против намеренно испорченного `sort`, он выбрал Vivado **2026.9** вместо **2026.10** и всего лишь напечатал предупреждение. Это

как раз та разновидность ошибки, из которой получается озадаченный студент и впустую потраченный вечер.

Claude тоже кое-что нашёл. Эти скрипты не выполняются, а подключаются через `source` в интерактивную оболочку пользователя, поэтому каждая переменная, которую забыли объявить как `local`, остаётся потом в вашей сессии. Claude это измерил: исходный скрипт оставлял после себя **семь** переменных, его собственная первая попытка — **одиннадцать**, а Codex — **ни одной**. После чего Claude исправил свой код, и вот это самое полезное.

И на одном поведении Claude настоял, а у Codex его до сих пор нет. Если каталог похож на установку Vivado, но его имя начинается не с цифры, Codex молча его пропускает. Claude его называет:

```
warning: ignoring '.../Xilinx/Vivado/mybuild', because the name
of a version directory is expected to start with a digit.
You can also use XILINX_VIVADO variable to specify a directory ...
```

Для учебного репозитория такое сообщение стоит реальных денег. Молчаливый вариант превращает исправление на пять секунд в ветку на форуме.

Второй раунд: они сошлись

А дальше произошло то, чего я не ожидал. Каждый агент прочитал разбор от другого и взял оттуда удачные идеи. За пару раундов оба перешли с `find` на шаблоны оболочки, оба стали требовать, чтобы имя каталога версии начиналось с цифры, оба начали искать `sort`, который действительно работает, оба отказались от поддержки установок без каталога версии, когда я решил, что этот случай нам не нужен, и оба пришли к одному и тому же стилю сообщений.

К последнему раунду Claude прогнал обе реализации через один стенд на одинаковых поддельных деревьях установки — включая совпадение версий в двух разных каталогах вендора, каталог версии, сделанный символьной ссылкой, и подменённый `sort` — и доложил, что в **одиннадцати случаях из одиннадцати** оба скрипта выбирают один и тот

же каталог. Две независимые реализации, ни строчки общего кода, одинаковый ответ каждый раз.

Когда два решения расходятся, одно из них вас чему-то учит. Когда они перестают расходиться — скорее всего, работа закончена.

Сравнивать осталось уже не поведение, а размер, диагностику и вкус:

ПОСЛЕ СБЛИЖЕНИЯ

	CODEx	CLAUDE	ИСХОДНЫЙ
Строк в файле	284	403	255
Функций	3	6	1
Переменных, оставленных в вашей оболочке	0	1	7
Объясняет, почему пропустил каталог	нет	да	нет
Выживает при подменённом sort	да	да	—
Расхождений в выборе из 11	0	0	—

Версия Codex на 119 строк короче, функций в ней вдвое меньше, и она не оставляет за собой переменных. Версия Claude — единственная, которая объясняет студенту, почему его установку проигнорировали. Это настоящий инженерный компромисс, и такое решение я с удовольствием приму сам — а именно туда я и хотел прийти.

Отчёты

Все шесть отчётов лежат в репозитории, по три с каждой стороны, написанные на последовательных этапах соревнования. Их стоит читать по порядку, потому что видно, как сокращается разрыв:

[misc/ai_sessions/20261003_xilinx_support](#)

CLAUDE [claude_report_1.html](#)

```
CLAUDE    claude_report_2.html
CLAUDE    claude_report_3.html
CODEX     codex_report_1.html
CODEX     codex_report_2.html
CODEX     codex_report_3.html
```

В каждом отчёте простым языком и по шагам разобрано, как именно выбирается установка Vivado, — а такой документации у проекта до этого просто не было, и появилась она как побочный результат соревнования.

Что я из этого вынес

Несколько вещей, в порядке важности — как я её вижу.

Соревнование показывает слепые зоны, которых не видит обзор кода

Если бы я попросил одного агента написать скрипт, а потом попросил его же проверить свою работу, я получил бы уверенный отчёт — и ошибку с виндовым `sort.exe` в придачу. Слабые места первой реализации стали видны именно из-за второй, потому что она сделала другой выбор, а не просто внимательнее посмотрела на тот же самый.

Заставляйте их тестировать, а не только писать

Самое ценное, что сделал каждый из агентов, — это стенд для тестов: заглушки вместо вспомогательных функций, поддельные деревья установки и прогон обоих скриптов по одинаковым случаям. Именно так ошибка с 2026.9 против 2026.10 перестала быть теорией. «Оно синтезируется» и «оно работает» — разные утверждения, и агент с радостью выдаст вам первое тоном, которым произносят второе.

Судить — навык меньший, чем писать, и он хорошо масштабируется

Я бы не написал по памяти ту проверку `sort`, которую придумал Codex. Но у меня не возникло ни малейших затруднений с тем, чтобы признать её лучше решения Claude,

когда оба лежали передо мной с пояснениями. Для человека, который поддерживает проект, в этой асимметрии и состоит вся возможность.

Сохраняйте отчёты

Положить отчёты со сравнением в `misc/ai_sessions/` оказалось полезнее, чем я думал. Через полгода, когда кто-нибудь спросит, зачем скрипт пробует три разных двоичных файла `sort`, ответ будет лежать в репозитории, а не в моей памяти.

Сейчас скрипт находит Vivado в каталогах `Xilinx`, `AMD` и `AMDDesignTools`, в обеих раскладках — старой и новой, правильно сравнивает номера версий и говорит что-то осмысленное, когда не находит ничего. Для этого понадобились два робота, спорящих друг с другом, и человек, решающий, кто из них прав.

`basics-graphics-music` находится по адресу github.com/yuri-panchul/basics-graphics-music. Число плат, список инструментов и приведённые в тексте цифры взяты из репозитория на момент написания.